OFFICIAL USER MANUAL

CXploit Framework

Official User Manual · Version 2.0

> Product: CXploit — Custom Exploit Development Framework
>
> Scope: 105 modules across 7 categories
>
> Platforms: Windows, Linux, macOS
>
> Requirement: Python 3.10 or later
>
> Repository: github.com/ravichandran3/cxploit-framework

Table of Contents

1 Introduction 1
1.1 Purpose and Scope of This Manual 1
1.2 Framework Overview 1
1.3 What Is New in Version 2.0 2
1.4 Legal and Ethical Use 3

2 System Architecture 4
2.1 Layered Architecture 4
2.2 Core Engine Components 5
2.3 Signature Capabilities 5

3 Installation 6
3.1 Prerequisites 6
3.2 Installing the Framework 6
Step 1 — Clone the repository 6
Step 2 — Create and activate a virtual environment 6
Step 3 — Install the dependencies 7
Step 4 — Verify the installation 7
3.3 Python Dependencies 7
3.4 Optional External Tools for Mobile Testing 8

4 Getting Started 8
4.1 Launching the Framework 8
4.2 Your First Scan 8
4.3 Automated Exploitation with AutoPwn 9
4.4 Resource Scripts and the REST API 10

5 Command-Line Reference 11
5.1 Navigation Commands 11
5.2 Module Commands 11
5.3 Execution Commands 12
5.4 Session Commands 12
5.5 Data Commands 13
5.6 Safety and Reporting Commands 13
5.7 Advanced Commands 14

6 Module Catalog 15
6.1 Reconnaissance (17 Modules) 15
6.2 Exploits (20 Modules) 16
6.3 Payloads (9 Modules) 18
6.4 Post-Exploitation (10 Modules) 18
6.5 Evasion (7 Modules) 19
6.6 Auxiliary (23 Modules) 20
6.7 C2, API, and Other Modules (19 Modules) 21

7 Sessions and Data Management 21
7.1 Session Lifecycle 21
7.2 Stored Data and the Database 22

8 Reporting and Compliance 22
8.1 Generating Reports 22
8.2 Compliance Reports 23
8.3 Report Contents 23

9 REST API Reference 23
9.1 Starting the API 24
9.2 Endpoints 24
9.3 Example: Running a Port Scan via the API 24

10 Configuration 25
10.1 Configuration File 25
10.2 Key Settings Explained 25

11 Extending the Framework 26
11.1 Plugin Architecture 26
11.2 Writing Your First Module 26
11.3 Coding Standards 28

12 Troubleshooting 28
12.1 Common Issues and Solutions 28
12.2 Enabling Debug Logging 29

Appendix A Authorized Use Guidelines 29

Appendix B Project Structure 30

Appendix C Release History 31
C.1 Version 2.0.0 (February 25, 2025) 31
C.2 Version 1.0.0 31

1 Introduction

1.1 Purpose and Scope of This Manual

This manual is the official user documentation for **CXploit Framework version 2.0**, a Metasploit-class exploit development and penetration testing framework written entirely in Python. It explains how to install, configure, operate, and extend the framework, and it serves as the authoritative reference for the interactive command-line interface, the module catalog, the REST API, and the reporting subsystem. The material is organized so that a reader can progress linearly from installation to advanced operation, while each chapter also stands alone for quick lookup during live engagements.

The primary audience is **security professionals and developers**: penetration testers, red-team operators, security researchers, and engineers who need an integrated testing environment. Readers are assumed to be familiar with command-line environments, basic networking concepts such as TCP ports and services, and the general workflow of a penetration test. No prior experience with CXploit is required; every workflow introduced in this manual is demonstrated with complete command-line examples.

Typographic conventions are used throughout. Text in a monospaced font, such as `recon/port_scanner` or `RHOST`, identifies commands, module paths, and option names exactly as they are typed in the console. Blocks with a shaded background and a blue left border reproduce console sessions, configuration files, and source code verbatim. Option names are written in upper case (for example `PORTS`) because the framework treats option names as case-sensitive identifiers.

1.2 Framework Overview

CXploit provides a modular, integrated environment for authorized security testing. At its core is a dynamic module loader that discovers **105 built-in modules across 7 categories** at startup and makes them available through a Metasploit-style interactive console with 72 commands, tab completion, module search, session management, and background jobs. An optional REST API and a real-time web dashboard expose the same engine for remote control and monitoring.

The framework covers the full lifecycle of an assessment:

- **Reconnaissance** — port scanning, host discovery, service enumeration, OS fingerprinting, web and vulnerability scanning, OSINT, DNS enumeration, SCADA/ICS and WiFi analysis, blockchain auditing, firmware analysis, supply-chain auditing, and offline CVE database matching.

- **Auto-exploitation** — an AI-powered AutoPwn pipeline that chains scanning, service detection, CVE matching, and automated exploitation behind a single command.
- **Exploitation** — SQL injection, XSS, LFI/RFI, SSH brute-forcing, buffer-overflow tooling, FTP/Telnet/RDP/VNC/SMTP exploit modules, database exploits for MySQL, MSSQL, PostgreSQL, Oracle, and Redis, browser and web-application attacks, IoT protocols, and multi-protocol credential spraying.
- **Payload generation** — reverse and bind shells, meterpreter-style stagers, polymorphic payloads with multi-layer encryption, shellcode generators, multi-handlers, and macro builders.
- **Post-exploitation** — system enumeration, persistence, credential harvesting, lateral movement, container escape, memory forensics, loot management, and Windows/Linux privilege-escalation helpers.
- **Evasion** — AES/XOR payload encryption, traffic obfuscation, timing randomization, WAF detection and bypass for more than 20 WAF products, AV/AMSI bypass, DNS tunneling, and JA3 fingerprint randomization.
- **Command and control** — an encrypted C2 server using AES-256 over TCP/HTTP channels with deployable agent stubs.
- **Reporting** — professional HTML, JSON, and PDF reports plus PCI-DSS, HIPAA, NIST, and GDPR compliance views with vulnerability severity ratings.

All results converge in a persistent SQLite database that records targets, services, vulnerabilities, credentials, sessions, and events, so that every action taken during a test remains queryable and reportable afterwards.

1.3 What Is New in Version 2.0

Version 2.0 is a major expansion of the framework. The module count grew from 36 to **105 modules (+69 new modules)**, and the CLI grew to 72 verified commands (19 base plus 53 extended). Table 1 summarizes the growth per category.

Table 1. Module growth from v1.0 to v2.0

Category	v1.0	v2.0	Highlights of new modules
Reconnaissance	7	17	WiFi, SCADA, OSINT, Blockchain, Firmware, Supply Chain, Wireless Protocol, Active Directory, Cloud, Threat Intel
Exploits	5	20	FTP, Telnet, RDP, VNC, SMTP, SNMP, LDAP, NFS, AD, Wireless, Cloud, MySQL, MSSQL, PostgreSQL, Oracle, Redis, Browser, WebApp, IoT

Category	v1.0	v2.0	Highlights of new modules
Payloads	6	9	Multi-Handler, Shellcode Generator, Macro Builder
Post-Exploitation	2	10	Lateral Movement, Container Escape, Memory Forensics, Loot Manager, Anti-Forensics, Ransomware Simulation, Windows and Linux Privilege Escalation
Evasion	2	7	AV Evasion (AMSI), Traffic Evasion (DNS tunnel, JA3), additional encryption modules
Auxiliary	2	23	Email Tester, HTTP Tunnel, FTP/HTTP/SSH/SSL/SNMP/NetBIOS/RDP/IPMI scanners, Credential Store, Report Generator
Total	36	105	+69 new modules

Version 2.0 also includes a series of quality improvements across the codebase: more than 88 bare `except:` clauses were replaced with proper `except Exception:` handling across 20+ files, a `check()` method was added to all module classes so targets can be tested without exploiting them, the paramiko TripleDES deprecation warning was suppressed, a duplicate banner display was fixed, and mobile-tester tool detection output was improved. All 153 Python files pass syntax verification with zero errors, and all 105 modules load, instantiate, and dispatch cleanly.

1.4 Legal and Ethical Use

This framework is designed for authorized security testing and educational purposes only. Unauthorized access to computer systems is illegal. Always obtain written permission before testing any system you do not own. The developers assume no liability for misuse of this software, and by using the tool you agree to comply with all applicable local, state, national, and international laws.

The project is licensed under the **PolyForm Strict License 1.0.0**. Under this license, noncommercial personal, educational, and research use is allowed, while redistribution and modification by anyone other than the copyright holder are not permitted. The repository consequently does not accept pull requests or external code contributions; bugs and security issues should be reported by opening an issue that describes the problem, without submitting a patch.

Professional rules of engagement apply to every engagement. Confirm that each target is inside the signed scope, use the framework's own safety features — target authorization, rate limiting, and the emergency stop — before running any module, and handle every credential,

session, and report artifact produced during testing as confidential data. Appendix A provides a practical pre-engagement checklist that operationalizes these obligations.

2 System Architecture

2.1 Layered Architecture

CXploit is organized as a layered architecture in which each layer only depends on the layers beneath it. The entry point `main.py` bootstraps the framework and can start the interactive CLI, a resource-script batch run, or the REST API server. The interface layer hosts the five user-facing surfaces; the core engine provides the services that every module shares; the modules layer contributes the 105 assessment modules; and a thin utilities layer supplies networking, cryptography, validation, and constants to all layers. Figure 1 shows the arrangement.

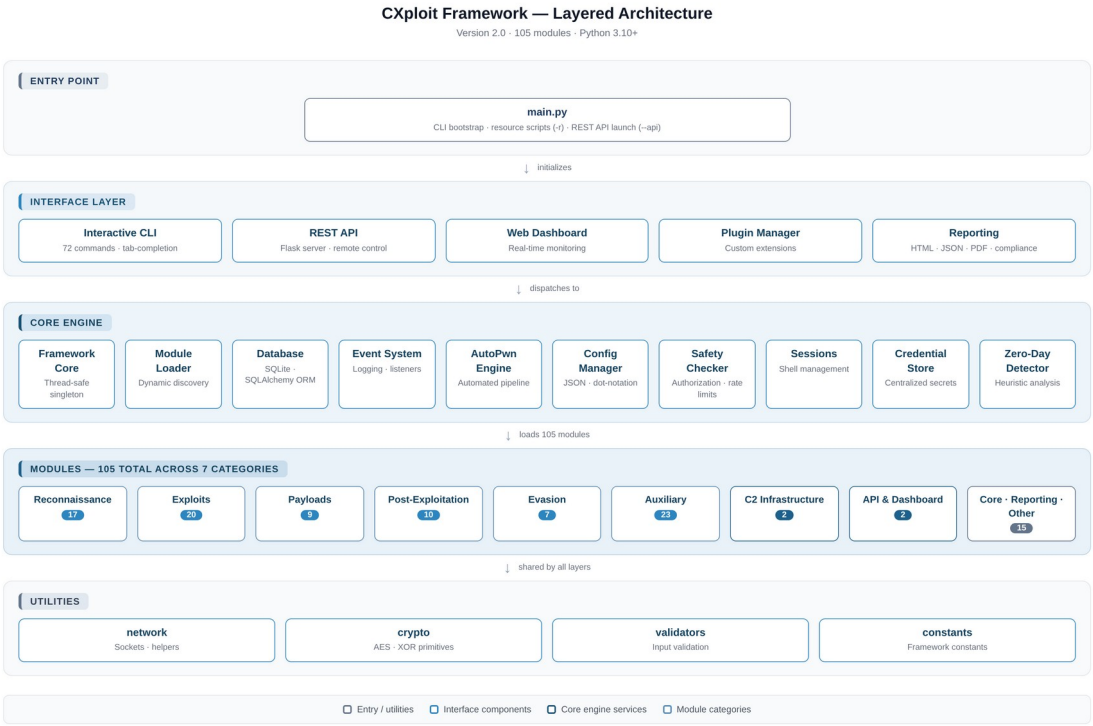


Figure 1. CXploit Framework layered architecture

This separation keeps the attack surface of the framework itself small and predictable: modules never talk to the database or the file system directly through ad-hoc code paths, but always through the core engine services. It also makes the framework extensible — a new module only needs to expose the standard class contract described in Chapter 11 and it is discovered, loaded, and dispatched automatically at the next startup.

2.2 Core Engine Components

The core engine is the backbone of the framework. Table 2 lists its principal components and the responsibility of each.

Table 2. Core engine components and their responsibilities

Component	Responsibility
105 built-in modules	Ready-to-use modules across 7 categories, available immediately after startup
Dynamic module loader	Auto-discovers all modules at startup using Python package introspection
Thread-safe singleton	Framework core uses a locked singleton pattern for safe concurrent access
SQLite database	Persistent storage for targets, services, vulnerabilities, credentials, sessions, and events
Context-managed sessions	All database operations use proper session scoping to prevent detached-instance errors
Event system	Centralized logging with file output and event listeners
Configuration manager	JSON-based configuration with dot-notation access and auto-merge with defaults
Safety system	Target authorization, emergency stop, and rate limiting
Plugin architecture	Extensible plugin system for custom functionality

2.3 Signature Capabilities

On top of the core engine, version 2.0 ships a set of capabilities that place CXploit in the same category as established exploitation frameworks. Each capability is exposed both as a module and, where relevant, as dedicated console commands.

Table 3. Metasploit-class capabilities introduced or matured in v2.0

Capability	Description
Auto-Exploitation (AutoPwn)	Automated scan, CVE match, and exploit pipeline with one command
Offline CVE database	50+ CVEs across 14 services with version-range CVSS matching
WAF detection and bypass	Fingerprints 20+ WAFs and auto-generates mutated SQLi/XSS bypass payloads

Capability	Description
Multi-protocol credential sprayer	Single module covering 11 protocols (SSH, FTP, HTTP, SMB, MySQL, RDP, and others)
Polymorphic payload engine	Self-modifying payloads with 5 mutation types and multi-layer encryption
Encrypted C2 infrastructure	AES-256 encrypted TCP/HTTP C2 server with deployable beacon agents
Real-time web dashboard	Live vulnerability heatmap, session monitor, and event log
Attack chain automation	Create, save, and replay multi-step attack sequences
SCADA/ICS scanning	Modbus, DNP3, BACnet, S7comm protocol scanning
IoT exploitation	MQTT, UPnP, and CoAP protocol attacks
Blockchain auditing	Smart contract analysis and DeFi vulnerability detection

3 Installation

3.1 Prerequisites

The framework requires **Python 3.10 or later** and has been tested on Python 3.10, 3.11, and 3.12. You also need **pip** (the Python package manager) and **Git** for cloning the repository. CXploit runs on Windows, Linux, and macOS; only the virtual-environment activation command differs between platforms, as shown below. No compiled components are required, and no external security tools need to be present for the framework to start.

3.2 Installing the Framework

Installation consists of four steps: cloning the repository, creating an isolated virtual environment, installing the dependencies, and verifying that the framework loads all modules.

Step 1 — Clone the repository

```
git clone https://github.com/ravichandran3/cxploit-framework.git
cd cxploit-framework
```

Step 2 — Create and activate a virtual environment

```
# Windows
python -m venv venv
.\venv\Scripts\activate
```

```
# Linux / macOS
python3 -m venv venv
source venv/bin/activate
```

Step 3 — Install the dependencies

```
pip install -r requirements.txt
```

Step 4 — Verify the installation

```
python main.py
```

A successful verification prints the CXploit banner and the confirmation line `Ready! 105 modules loaded.` If the module count differs from 105, re-check that the virtual environment is active and that the dependency installation completed without errors; Chapter 12 covers the remaining failure modes in detail.

3.3 Python Dependencies

The table below lists every package pinned in `requirements.txt` and the role each one plays in the framework. All of them are installed automatically by the command in Step 3.

Table 4. Pinned Python dependencies

Package	Version	Purpose
`scapy`	2.5.0	Raw packet crafting and network analysis
`paramiko`	3.4.0	SSH protocol implementation
`requests`	2.31.0	HTTP client for web exploits
`beautifulsoup4`	4.12.2	HTML parsing for web scanning
`colorama`	0.4.6	Cross-platform terminal colors
`prompt-toolkit`	3.0.43	Interactive CLI with tab completion
`pycryptodome`	3.20.0	AES, XOR, and other encryption
`jinja2`	3.1.3	HTML report templating
`reportlab`	4.1.0	PDF report generation
`flask`	3.0.2	REST API server and dashboard
`python-nmap`	0.7.1	Nmap integration for scanning
`rich`	13.7.0	Rich terminal formatting and tables
`aiohttp`	3.9.3	Asynchronous HTTP operations
`sqlalchemy`	2.0.25	Database ORM and session management

3.4 Optional External Tools for Mobile Testing

Six external tools are **not required** to install or run the framework, but they extend the mobile-testing commands (see the `mobile_test` command in Chapter 5). When a tool is absent, the relevant command reports it as "Not installed" and the remaining functionality is unaffected.

Table 5. Optional external tools that enhance mobile testing

Tool	Purpose	Installation
`adb`	Android Debug Bridge	developer.android.com/studio
`apktool`	APK reverse engineering	github.com/iBotPeaches/Apktool
`jadx`	DEX to Java decompiler	github.com/skylot/jadx
`frida`	Dynamic instrumentation	pip install frida-tools
`objection`	Mobile exploration toolkit	pip install objection
`mitmproxy`	SSL/TLS interception	pip install mitmproxy

4 Getting Started

4.1 Launching the Framework

Start the interactive console from the repository root:

```
python main.py
```

The startup sequence loads the configuration, opens the SQLite database, discovers every module, and then presents the `cxploit >` prompt together with the banner. The console built on `prompt_toolkit` provides context-aware tab completion for commands, module names, and option names, so pressing the Tab key after a partial command lists every valid continuation.

4.2 Your First Scan

The fastest way to become productive is a complete reconnaissance pass against a lab host. The following session locates the port scanner, selects it, configures its options, runs it, and inspects the recorded results. Note how the prompt changes to reflect the selected module.

```
cxploit > search scan
  recon/port_scanner    -- TCP/UDP port scanner with service detection
  recon/vuln_scanner    -- Vulnerability scanner
  recon/web_scanner     -- Web application scanner
```

```
  recon/scada_scanner   -- SCADA/ICS scanner
  recon/wifi_scanner    -- WiFi scanner

cxploit > use recon/port_scanner
cxploit (recon/port_scanner) > show options
cxploit (recon/port_scanner) > set RHOST 192.168.1.1
cxploit (recon/port_scanner) > set PORTS 1-1024
cxploit (recon/port_scanner) > set THREADS 100
cxploit (recon/port_scanner) > run

  [+] 22/tcp OPEN  (ssh)
  [+] 80/tcp OPEN  (http) | Apache/2.4.52
  [+] 443/tcp OPEN (https)

cxploit (recon/port_scanner) > back
cxploit > show targets
cxploit > show vulns
cxploit > report html
```

Every discovered host, service, and vulnerability is written to the database as soon as it is found, which is why `show targets` and `show vulns` return data even after the module has been deselected. The `report html` command finally converts the recorded findings into a standalone HTML report (Chapter 8).

4.3 Automated Exploitation with AutoPwn

The AutoPwn engine compresses the entire assessment workflow into a single command. Given a host or a CIDR range, it runs the four-phase pipeline shown in Figure 2: scanning, vulnerability matching against the offline CVE database, automated exploitation of confirmed weaknesses, and data capture with report generation.

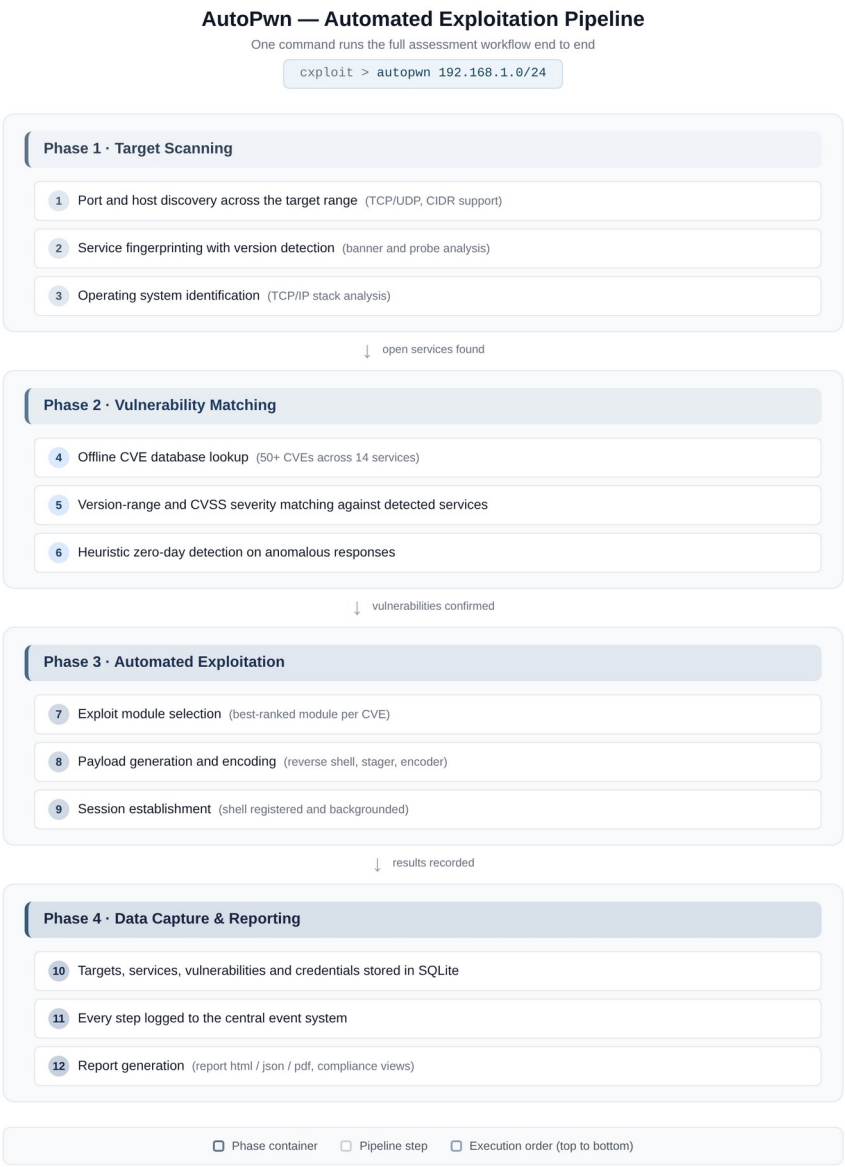


Figure 2. The AutoPwn automated exploitation pipeline

Launch the pipeline with the `autopwn` command followed by the target:

```
cxploit > autopwn 192.168.1.0/24
```

Because the pipeline can attempt exploitation automatically, run it only against targets that are explicitly authorized. The safety system described in Section 2.2 — target authorization, rate limiting, and the emergency stop — applies to every phase of the pipeline.

4.4 Resource Scripts and the REST API

Routine workflows can be stored in resource scripts and replayed non-interactively. A resource script is a plain text file (conventionally with the `.rc` extension) containing one console command per line, exactly as it would be typed at the prompt:

```
python main.py -r scripts/my_scan.rc
```

For remote control and integration with other tooling, start the framework together with its REST API server. The API listens on the port you specify and exposes the endpoints documented in Chapter 9:

```
python main.py --api --api-port 8080
```

5 Command-Line Reference

The interactive console implements **72 commands** — 19 base commands plus 53 extended commands covering mobile testing, SCADA scanning, WiFi analysis, threat intelligence, compliance, and more. Commands are grouped below by function. All command names are lower case, and options set with `set` are upper-case identifiers. Where a command supports shortcuts (for example `?` for `help`), both forms are listed.

5.1 Navigation Commands

These commands control the console itself and provide general framework information.

Table 6. Navigation commands

Command	Description
`help`	Show all available commands
`banner`	Display the framework banner
`clear`	Clear the screen
`exit`	Shutdown and exit the framework
`stats`	Show framework statistics

The aliases `?` (for `help`), `sysinfo` (for `stats`), and `quit` (for `exit`) behave identically to their long forms.

5.2 Module Commands

Module commands select, inspect, and search the 105 loaded modules. Module selection supports partial names, so unique prefixes such as `use recon/port` resolve to the full module path.

Table 7. Module commands

Command	Description
`use <module>`	Select a module (supports partial names)
`back`	Deselect the current module
`info`	Show detailed module information
`show modules`	List all 105 loaded modules
`show options`	Show the current module's configurable options
`search <query>`	Search modules by name or description
`reload`	Reload the current module from disk

5.3 Execution Commands

Execution commands configure the current module, run it in the foreground or background, and manage background jobs. The `check` command is the safe counterpart of `run`: it determines whether the target is vulnerable without exploiting it.

Table 8. Execution commands

Command	Description
`set <OPT> <VALUE>`	Set a module option
`setg <OPT> <VALUE>`	Set a global option (persists across modules)
`unset <OPT>`	Clear a module option
`run`	Execute the current module
`bg`	Run the module in a background thread
`check`	Check if target is vulnerable (without exploiting)
`jobs`	List background jobs
`jobs -k <id>`	Kill a background job

The aliases `exploit` (for `run`) and `run -j` (for `bg`) are provided for familiarity with established frameworks.

5.4 Session Commands

Sessions represent active connections to compromised hosts, such as a reverse shell opened by a payload. The session commands list, interact with, and terminate sessions.

Table 9. Session commands

Command	Description
`sessions -l`	List all sessions
`sessions -i <id>`	Interact with a session (enter shell)
`sessions -k <id>`	Kill a specific session
`sessions -K`	Kill all sessions

5.5 Data Commands

The data commands query the SQLite database that the framework maintains. Every target, vulnerability, credential, session, and event recorded during the engagement can be retrieved here without leaving the console.

Table 10. Data commands

Command	Description
`show targets`	Show all targets in the database
`show vulns`	Show discovered vulnerabilities
`show creds`	Show harvested credentials
`show sessions`	Show exploitation sessions
`show events`	Show recent event logs
`show stats`	Show framework statistics

5.6 Safety and Reporting Commands

These commands govern the framework's safety mechanisms and generate the deliverable reports of an engagement.

Table 11. Safety and reporting commands

Command	Description
`authorize <target>`	Authorize a target for testing
`emergency_stop`	Halt all module execution (toggle)
`scan <target>`	Quick port scan shortcut
`report <format>`	Generate a report (html/json/pdf)

Command	Description
`compliance <framework>`	Generate a compliance report (pci/hipaa/nist/gdpr)

5.7 Advanced Commands

The extended command set exposes the specialized engines of the framework directly from the console, from automated exploitation to niche protocol analysis.

Table 12. Advanced commands

Command	Description
`autopwn <target/CIDR>`	Run the full auto-exploitation pipeline
`dashboard [port]`	Launch the real-time web dashboard
`cve_search <service> [version]`	Search the offline CVE database
`chain create <name>`	Create a named attack chain
`chain add <command>`	Add a step to the current chain
`chain run [name]`	Execute an attack chain
`chain list`	List all defined chains
`chain save <file>`	Save chains to a JSON file
`mobile_test [mode]`	Mobile application security testing
`scada_scan <target>`	SCADA/ICS protocol scanning
`wifi_scan [interface]`	WiFi network scanning
`threat_intel <target>`	Threat intelligence gathering
`supply_chain <project_dir>`	Supply chain security audit
`firmware <file>`	Firmware analysis
`blockchain <address>`	Blockchain/smart contract auditing

Command	Description
`osint <domain>`	OSINT intelligence gathering

6 Module Catalog

This chapter is the complete reference of the **105 modules** shipped with CXploit 2.0, organized by category exactly as the console's `show modules` command reports them. Module paths are relative to the framework and are used with the `use` command; descriptions are reproduced from the module metadata. Categories cover reconnaissance, exploitation, payload generation, post-exploitation, evasion, auxiliary tooling, and the remaining infrastructure modules such as C2, API, and dashboard services.

6.1 Reconnaissance (17 Modules)

Reconnaissance modules collect information about the target environment before any exploitation is attempted. They range from classic port scanning to niche audits such as firmware analysis and smart-contract review.

Table 13. Reconnaissance modules

Module	Description
`recon/port_scanner`	TCP/UDP port scanner with CIDR support, 5 timing profiles, OS fingerprinting
`recon/host_discovery`	Network host discovery using ARP, ICMP, TCP SYN
`recon/service_enum`	Deep service fingerprinting with version detection
`recon/os_fingerprint`	OS identification via TCP/IP stack analysis
`recon/web_scanner`	Web application scanner with directory brute-forcing and tech detection
`recon/vuln_scanner`	Automated vulnerability detection with CVE matching
`recon/cve_db`	Offline CVE database — 50+ CVEs across 14 services
`recon/dns_enum`	DNS enumeration — zone transfers, subdomain brute, wildcard detection
`recon/osint`	OSINT intelligence — WHOIS, DNS, subdomains, emails, certificate transparency
`recon/network_mapper`	Network mapping — ARP discovery, OS detection, topology visualization

Module	Description
recon/wifi_scanner	WiFi scanning — SSID discovery, encryption analysis, rogue AP detection
recon/scada_scanner	SCADA/ICS scanning — Modbus, DNP3, BACnet, S7comm, EtherNet/IP
recon/threat_intel	Threat intelligence — IOC analysis, STIX/TAXII, feed aggregation
recon/supply_chain_auditor	Supply chain audit — dependency CVE scanning, typosquat detection
recon/firmware_analyzer	Firmware analysis — binary extraction, string analysis, crypto detection
recon/blockchain_auditor	Blockchain audit — smart contract analysis, DeFi vulnerability detection
recon/wireless_protocol	Wireless protocol analyzer — Bluetooth, Zigbee, LoRa, NFC, Z-Wave

6.2 Exploits (20 Modules)

Exploit modules actively test and, where authorized, exercise weaknesses across web applications, network services, database engines, browsers, and specialized platforms such as IoT devices.

Table 14. Exploit modules

Module	Description
exploit/web/sqli	SQL injection — error/blind/time/union techniques, multi-DBMS
exploit/web/xss	Cross-site scripting — reflected, stored, DOM-based with WAF bypass
exploit/web/lfi_rfi	Local/Remote file inclusion with PHP wrappers and log poisoning
exploit/web/advanced_web_exploit	Advanced web — deserialization, SSRF, SSTI attacks
exploit/network/ssh_brute	SSH brute-force with wordlist support and multi-threading
exploit/network/ftp_exploit	FTP exploitation — anonymous login, path traversal, command injection

Module	Description
`exploit/network/telnet_exploit`	Telnet exploitation — default credentials, command injection
`exploit/network/rdp_exploit`	RDP exploitation — BlueKeep detection, brute-force, NLA bypass
`exploit/network/vnc_exploit`	VNC exploitation — auth bypass, brute-force, screenshot capture
`exploit/network/smtp_exploit`	SMTP exploitation — open relay, user enumeration, header injection
`exploit/network/snmp_exploit`	SNMP exploitation — community string brute-force, MIB walking
`exploit/network/ldap_exploit`	LDAP exploitation — anonymous bind, injection, password spraying
`exploit/network/nfs_exploit`	NFS exploitation — share enumeration, mount access, UID spoofing
`exploit/network/ad_exploit`	Active Directory — Kerberoasting, AS-REP roasting, DCSync
`exploit/network/wireless_exploit`	Wireless — WEP/WPA cracking, evil twin, deauth attacks
`exploit/network/cloud_exploit`	Cloud — S3 bucket enumeration, metadata service, IAM misconfiguration
`exploit/database/mysql_exploit`	MySQL — UDF injection, file read/write, privilege escalation
`exploit/database/mssql_exploit`	MSSQL — xp_cmdshell, linked servers, privilege escalation
`exploit/database/postgresql_exploit`	PostgreSQL — COPY command RCE, large object abuse
`exploit/database/oracle_exploit`	Oracle — TNS poisoning, Java stored procedure RCE
`exploit/database/redis_exploit`	Redis — unauthorized access, Lua sandbox escape, module loading
`exploit/bof/stack_overflow`	Buffer overflow — 5-stage workflow (fuzz/pattern/offset/badchars/exploit)
`exploit/browser/browser_exploit`	Browser exploitation — JS injection, WebRTC leak, credential stealing
`exploit/specialized/webapp_exploit`	WebApp advanced — deserialization chains, SSRF, SSTI

Module	Description
`exploit/specialized/iot_exploit`	IoT — MQTT exploitation, UPnP attacks, CoAP manipulation

6.3 Payloads (9 Modules)

Payload modules generate and manage the code that runs on a target after a successful exploit, from simple reverse shells to polymorphic, multi-layer-encrypted stagers.

Table 15. Payload modules

Module	Description
`payload/reverse_shell`	Reverse shell multi-handler with interactive I/O
`payload/bind_shell`	Bind shell connector
`payload/meterpreter`	Meterpreter-style stager with 12+ built-in commands
`payload/encoder`	Payload encoder — XOR, Base64, custom schemes
`payload/generator`	Platform-specific payload generator
`payload/polymorphic`	Polymorphic payload engine — 5 mutation types, multi-layer encryption
`payload/multi_handler`	Multi-handler — manages multiple simultaneous payload connections
`payload/shellcode_generator`	Shellcode generator — x86/x64 shellcode for Windows/Linux
`payload/macro_builder`	Macro builder — Office VBA/HTA/JS payload generation

6.4 Post-Exploitation (10 Modules)

Post-exploitation modules operate on hosts where access has already been obtained, enumerating systems, elevating privileges, collecting data, and measuring the resilience of forensic trails.

Table 16. Post-exploitation modules

Module	Description
`post/sys_enum`	System enumeration — OS, users, software, network, processes
`post/persistence`	Persistence — registry, cron, services, scheduled tasks

Module	Description
`post/lateral_move`	Lateral movement — WMI, PsExec, SSH pivoting, pass-the-hash
`post/container_escape`	Container escape — Docker socket, privileged mode, kernel exploits
`post/memory_forensics`	Memory forensics — process dumping, credential extraction, DLL analysis
`post/loot_manager`	Loot manager — file search, credential harvesting, data exfiltration
`post/antiforensics`	Anti-forensics — log cleaning, timestamp modification, trace removal
`post/ransom_sim`	Ransomware simulation — safe encryption/decryption demo for training
`post/windows_post`	Windows post-exploitation — Mimikatz integration, UAC bypass, token manipulation
`post/linux_post`	Linux post-exploitation — privilege escalation, container detection, persistence

6.5 Evasion (7 Modules)

Evasion modules reduce the detectability of payloads and traffic. They are grouped under the encryption, WAF-bypass, and traffic-evasion families of the framework.

Table 17. Evasion modules

Module	Description
`evasion/encryption`	Payload encryption — AES-256 and XOR with self-decrypting stubs
`evasion/encryption/TrafficObfuscator`	Traffic obfuscation — user-agent rotation, domain fronting, jitter
`evasion/encryption/AntiForensics`	Anti-forensics — log cleaning, timestomping scripts
`evasion/encryption/TimingRandomizer`	Timing randomization — evade behavior-based detection
`evasion/waf_bypass`	WAF bypass — fingerprints 20+ WAFs, generates mutated payloads
`evasion/av_evasion`	AV evasion — AMSI bypass, obfuscation, process hollowing

Module	Description
evasion/traffic_evasion	Traffic evasion — DNS tunneling, JA3 fingerprint randomization

6.6 Auxiliary (23 Modules)

Auxiliary modules provide supporting tooling: protocol scanners, credential handling, tunneling, wordlist generation, and report utilities that do not fit the exploit or recon families.

Table 18. Auxiliary modules

Module	Description
auxiliary/tools/HTTPFileServer	HTTP file server for payload delivery
auxiliary/tools/DNSLookup	DNS lookup and enumeration
auxiliary/tools/HashCracker	Hash cracker — MD5, SHA1, SHA256, brute-force
auxiliary/tools/WordlistGenerator	Wordlist generator with mutations and leet speak
auxiliary/cred_sprayer	Multi-protocol credential sprayer (11 protocols)
auxiliary/email_tester	Email security tester — SPF, DKIM, DMARC, open relay
auxiliary/http_tunnel	HTTP tunneling for firewall evasion
auxiliary/hash_cracker	Advanced hash cracker with rainbow tables
auxiliary/ftp_scanner	FTP vulnerability scanner — anonymous login, version detection
auxiliary/http_scanner	HTTP vulnerability scanner — headers, methods, misconfigurations
auxiliary/ssh_scanner	SSH scanner — algorithm enumeration, version detection
auxiliary/ssl_scanner	SSL/TLS scanner — certificate validation, cipher analysis, HSTS
auxiliary/snmp_scanner	SNMP scanner — community strings, MIB walking
auxiliary/netbios_scanner	NetBIOS scanner — name resolution, share enumeration
auxiliary/rdp_scanner	RDP scanner — NLA detection, version fingerprinting
auxiliary/ipmi_scanner	IPMI scanner — cipher 0 detection, hash extraction
auxiliary/credential_store	Credential store — centralized credential management
auxiliary/report_generator	Report generator — HTML/JSON/PDF output

6.7 C2, API, and Other Modules (19 Modules)

The remaining modules form the infrastructure of the framework: the encrypted command-and-control server and its agent generator, the REST API and dashboard services, the core engines exposed as modules, the reporting pair, and five additional reconnaissance modules. The category ships **19 modules** in total; the principal ones are listed below.

Table 19. C2, API, and other infrastructure modules

Group	Modules
C2 Infrastructure	`c2/c2_server` — Encrypted C2 server with AES-256
C2 Infrastructure	`c2/c2_agent` — Deployable beacon generator
API & Dashboard	`api/server` — Flask REST API
API & Dashboard	`api/dashboard` — Real-time web dashboard
Core	`core/autopwn` — Auto-exploitation engine
Core	`core/credential_store` — Centralized credential store
Core	`core/report_generator` — Report generator service
Core	`core/zeroday_detector` — Zero-day detection engine
Reporting	`reporting/report_gen` — HTML/JSON/PDF report generation
Reporting	`reporting/compliance_report` — Compliance report generation
Recon (additional)	`recon/mobile_tester` — Mobile application security testing
Recon (additional)	`recon/ad_enum` — Active Directory enumeration
Recon (additional)	`recon/bluetooth_scanner` — Bluetooth device scanning
Recon (additional)	`recon/cert_analyzer` — Certificate transparency analysis
Recon (additional)	`recon/cloud_scanner` — Cloud environment scanning

7 Sessions and Data Management

7.1 Session Lifecycle

A session is created whenever a payload successfully connects back to the framework — for example, a reverse shell opened by an exploit module. The console announces the session

with an identifier, and the `sessions` family of commands manages it from listing to termination. The following transcript shows a complete lifecycle.

```
# After a successful exploit with a reverse shell:
[+] Session created: a1b2c3d4

# List sessions
cxploit > sessions -l

# Interact with a session
cxploit > sessions -i a1b2c3d4
[*] Interactive session a1b2c3d4. Press Ctrl+C to background.

$ whoami
root
$ ^C
[*] Session a1b2c3d4 backgrounded

# Kill a session
cxploit > sessions -k a1b2c3d4
```

Pressing Ctrl+C inside an interactive session backgrounds it instead of closing it, so several sessions can be kept open side by side and revisited with `sessions -i`. Killing a session with `sessions -k` closes the connection and removes it from the list, while `sessions -K` tears down every session at once — the recommended action at the end of an engagement.

7.2 Stored Data and the Database

All structured findings live in the SQLite database managed by the core engine. Targets, services, vulnerabilities, credentials, sessions, and events are recorded as they are discovered, and the `show` family of commands (Table 10) queries them at any time. Because the data persists between runs, a scan interrupted on one day can be reviewed and reported on the next without repeating any network activity.

The credential store deserves particular attention during engagements: harvested credentials are centralized there and can be reused by modules such as the multi-protocol credential sprayer. Treat the database file itself as sensitive engagement data and remove it once the reporting obligations of the engagement are complete.

8 Reporting and Compliance

8.1 Generating Reports

Professional reports are generated directly from the console with the `report` command. Three formats are available: a dark-themed HTML report for interactive review, machine-readable JSON for downstream tooling, and a PDF document for formal delivery.

```
cxploit > report html       # HTML report with dark theme
cxploit > report json       # Machine-readable JSON
cxploit > report pdf        # PDF document
```

Reports are saved to `data/reports/` using the naming scheme `report_YYYYMMDD_HHMMSS.<format>`, so successive reports of the same engagement never overwrite each other.

8.2 Compliance Reports

For engagements that must demonstrate regulatory coverage, the `compliance` command maps the recorded findings to the requirements of four widely used frameworks:

```
cxploit > compliance pci     # PCI-DSS compliance report
cxploit > compliance hipaa   # HIPAA compliance report
cxploit > compliance nist    # NIST compliance report
cxploit > compliance gdpr    # GDPR compliance report
```

8.3 Report Contents

Every report contains an executive summary with the target, vulnerability, and credential counts, followed by the detailed sections. Vulnerabilities are rated on the standard severity scale from **Critical** through **High** and **Medium** to **Low**, and each entry carries its description, associated CVE identifier where applicable, and proof of the finding. The compliance views add a mapping section that links each finding to the specific requirements of the chosen framework.

Table 20. Sections of a generated report

Section	Content
Executive Summary	Target count, vulnerability count, credential count
Targets	IP, hostname, OS, status
Vulnerabilities	Name, severity (Critical/High/Medium/Low), description, CVE, proof
Credentials	Service, username, password/hash
Compliance Mapping	Framework-specific requirement coverage

9 REST API Reference

9.1 Starting the API

The framework includes a Flask-based REST API for remote control and integration with external tooling such as SIEM dashboards, ticketing systems, or custom front ends. Start the framework with the API flags to expose it:

```
python main.py --api --api-host 0.0.0.0 --api-port 5000
```

The API version prefix is `/api/v1`. Every endpoint returns JSON, and module execution requests accept their options as a JSON body in which option names are written in lower case.

9.2 Endpoints

Table 21. REST API endpoints (v1)

Method	Endpoint	Description
GET	`/api/v1/status`	Framework status and version
GET	`/api/v1/modules`	List all loaded modules
GET	`/api/v1/modules/<name>`	Get module details and options
POST	`/api/v1/modules/<name>/run`	Execute a module with options
GET	`/api/v1/targets`	List all targets in database
GET	`/api/v1/vulnerabilities`	List all discovered vulnerabilities
GET	`/api/v1/sessions`	List active sessions
POST	`/api/v1/report`	Generate a report

9.3 Example: Running a Port Scan via the API

The following request executes the port scanner with its options supplied in the JSON body, exactly mirroring the interactive workflow from Section 4.2:

```
curl -X POST http://localhost:5000/api/v1/modules/recon/port_scanner/run \
  -H "Content-Type: application/json" \
  -d '{"rhost": "192.168.1.1", "ports": "1-1024", "threads": "50"}'
```

Read-only endpoints follow the same pattern: `GET /api/v1/targets` returns the recorded targets, and `POST /api/v1/report` generates a report in the requested format. Because the API can trigger active testing, bind it to an interface and port that are reachable only by authorized operators.

10 Configuration

10.1 Configuration File

Configuration is stored in `data/config.json` and is auto-created with sensible defaults on first launch. The file is merged with the built-in defaults at startup, so a file that sets only a single key keeps every other default intact. The dot-notation configuration manager in the core engine reads these values, which is why settings are documented here as dotted paths.

```
{
    "framework": {
        "max_threads": 10,
        "default_timeout": 5,
        "loglevel": "INFO"
    },
    "scanner": {
        "scan_timeout": 3,
        "max_concurrent_scans": 50,
        "default_ports": "1-1024",
        "service_detection": true
    },
    "payload": {
        "default_lhost": "0.0.0.0",
        "default_lport": 4444,
        "encoder": "xor",
        "iterations": 1
    },
    "reporting": {
        "output_dir": "data/reports",
        "default_format": "html",
        "include_evidence": true
    },
    "safety": {
        "require_authorization": false,
        "rate_limit_rps": 100,
        "auto_backup": true
    }
}
```

10.2 Key Settings Explained

The five sections of the configuration file govern the framework's concurrency, scanning behavior, payload defaults, report output, and safety rails. Table 21 summarizes the settings that operators most frequently adjust.

Table 22. Frequently adjusted configuration settings

Setting	Default	Effect
`framework.max_threads`	10	Upper bound on worker threads used by modules
`framework.loglevel`	INFO	Log verbosity; set to DEBUG for troubleshooting
`scanner.max_concurrent_scans`	50	Parallel scan operations allowed at once
`scanner.default_ports`	1-1024	Port range used when a module does not override PORTS
`payload.default_lhost`	0.0.0.0	Default listen address for handlers
`payload.default_lport`	4444	Default listen port for handlers
`reporting.output_dir`	data/reports	Directory where generated reports are written
`reporting.default_format`	html	Format used when report is called without an argument
`safety.require_authorization`	false	When true, targets must be authorized before testing
`safety.rate_limit_rps`	100	Requests-per-second ceiling enforced by the safety system

On multi-user or shared systems, consider enabling `safety.require_authorization` so that every target must pass through the `authorize` command before any module will touch it. Combined with `rate_limit_rps` and the `emergency_stop` toggle, this provides a three-layer guardrail around all automated activity.

11 Extending the Framework

11.1 Plugin Architecture

CXploit is designed to grow with your needs. Custom functionality is added by dropping Python files into the framework's module directories (or the `plugins/custom/` area); the dynamic module loader auto-discovers every conforming class on the next startup. There is no registration step, no configuration change, and no rebuild — the module simply appears in `show modules` and becomes searchable with `search`.

11.2 Writing Your First Module

A module is a Python class that defines three members: a `MODULE_INFO` dictionary with metadata, an `options` dictionary declaring every configurable parameter, and a `run()` method containing the module logic. Adding a `check()` method — the standard across all built-in modules — lets operators test whether a target is affected without running the full module. The listing below is the official minimal example from the project documentation.

```
# exploit_framework/recon/my_scanner.py

class MyScanner:
    MODULE_INFO = {
        "name": "My Scanner",
        "description": "Scans for something specific",
        "author": "Your Name",
        "rank": "good",
    }

    def __init__(self):
        self.framework = None
        self.options = {
            "RHOST": {"value": None, "required": True,
                      "description": "Target host"},
            "RPORT": {"value": "80", "required": False,
                      "description": "Target port"},
        }

    def run(self):
        host = self.options["RHOST"]["value"]
        port = int(self.options["RPORT"]["value"])
        print(f"[+] Scanning {host}:{port}")

        if self.framework and hasattr(self.framework, "db"):
            self.framework.db.add_target(host, status="up")

        return {"host": host, "port": port, "status": "complete"}

    def check(self):
        """Quick vulnerability check."""
        return True
```

Two details in the example are worth internalizing. First, the module never hard-codes its parameters: every value comes from `self.options`, which is what makes `show options` and `set` work uniformly across all 105 modules. Second, results are stored through the framework handle — `self.framework.db.add_target()` — rather than only printed, so the finding becomes queryable and appears in reports.

To verify that a new module loads cleanly together with the rest of the framework, run the built-in verification snippet:

```
python -c "
from exploit_framework.core.framework import Framework
fw = Framework()
fw.initialize()
print(f'Modules: {fw.modules.count()}')
print(f'Stats: {fw.get_stats()}')
fw.shutdown()
"
```

11.3 Coding Standards

Contributions to the project's codebase follow a short list of firm standards, and custom modules are expected to adhere to them as well:

- PEP 8 compliance for style.
- Docstrings on all public methods.
- Use `except Exception:` — never a bare `except:`.
- No hard-coded values — use `self.options` for configurable parameters.
- Store results in the database when possible.

Note that the project is licensed under PolyForm Strict 1.0.0, which does not permit redistribution or modification by anyone other than the copyright holder; the repository therefore does not accept pull requests. Bugs and security issues should be reported by opening an issue that describes the problem.

12 Troubleshooting

12.1 Common Issues and Solutions

Most operational problems fall into a small set of known categories: environment activation, module discovery, port conflicts, operating-system permissions, and the framework's own safety toggles. Table 22 lists the documented issues together with their solutions.

Table 23. Documented troubleshooting matrix

Issue	Solution
`ModuleNotFoundError`	Ensure the venv is activated and pip install -r requirements.txt completed
`0 modules loaded`	Check that exploit_framework/__init__.py exists and sys.path is correct
`Port already in use`	Another process is using the port; use netstat -tlnp to find it
`Permission denied (scanning)`	Some scans require root/admin privileges (especially UDP, ICMP)

Issue	Solution
`Emergency stop active`	Run emergency_stop again to toggle it off
`Target not authorized`	Run authorize <target> before testing
`Mobile tools "Not installed"`	Optional external tools — see Section 3.4 for installation instructions

12.2 Enabling Debug Logging

When behavior cannot be explained by the table above, raise the log level to **DEBUG** in `data/config.json` and restart the framework:

```
{
    "framework": {
        "loglevel": "DEBUG"
    }
}
```

Log files are written to `data/logs/`. The event stream is the same one exposed through `show events` in the console, so recent activity can be inspected interactively without leaving the framework.

Appendix A Authorized Use Guidelines

This appendix converts the legal and ethical obligations of Section 1.4 into a practical pre-engagement routine. Work through every item before running any module; the framework's safety features assume an operator who has already completed this diligence.

- **Written authorization.** Obtain a signed statement of work, rules-of-engagement document, or equivalent written permission that names the systems, networks, and applications in scope before testing any system you do not own.
- **Scope discipline.** Confirm that every target address, domain, and account you configure matches the authorized scope — including the ranges you pass to autopwn, which expand across entire subnets.
- **Target authorization.** Register each in-scope target with the authorize command (and consider setting safety.require_authorization to true, Section 10.2) so unregistered systems are refused.
- **Rate awareness.** Keep the safety.rate_limit_rps ceiling and module THREADS options at levels the target environment can absorb; aggressive timing profiles can disrupt production services.

- **Emergency readiness.** Know the emergency_stop toggle before you need it: running emergency_stop once halts all module execution, and running it again re-enables activity.
- **Data handling.** Treat credentials, sessions, loot, reports, and the SQLite database as confidential engagement data; store, transmit, and dispose of them according to your client's data-handling requirements.
- **Legal compliance.** Comply with all applicable local, state, national, and international laws; unauthorized access to computer systems is illegal regardless of intent.

The developers assume no liability for misuse of the framework. Staying inside an authorized scope protects you, your client, and the continued existence of open security tooling.

Appendix B Project Structure

The repository layout mirrors the layered architecture of Chapter 2: one directory per module family, a core package for engine services, and a data directory for runtime artifacts. The condensed tree below shows every major path; module counts per directory are annotated.

```
cxploit-framework/
|-- main.py                    # Entry point
|-- requirements.txt           # Python dependencies
|-- README.md                  # Project documentation
|-- exploit_framework/
|   |-- __init__.py            # Package init + BANNER
|   |-- api/                   # server.py, dashboard.py
|   |-- auxiliary/             # 23 modules (scanners, tools, cred store)
|   |-- c2/                    # c2_server.py, c2_agent.py (AES-256 C2)
|   |-- cli/                   # commands.py, completer.py, console.py
|   |-- core/                  # framework, autopwn, database, config,
|   |                          #   events, module_loader, session,
|   |                          #   credential_store, report_generator,
|   |                          #   zeroday_detector
|   |-- evasion/               # 7 modules (encryption, WAF, AV, traffic)
|   |-- exploits/              # 20 modules (web, network, database,
|   |                          #   bof, browser, specialized)
|   |-- payloads/              # 9 modules (shells, stager, polymorphic...)
|   |-- post/                  # 10 modules (enum, persistence, lateral...)
|   |-- recon/                 # 17 modules (scan, osint, scada, wifi...)
|   |-- reporting/             # report_gen.py, compliance_report.py
|   +-- utils/                 # constants, crypto, network, validators
|-- data/                      # Runtime data
|   |-- config.json            # Configuration
|   |-- logs/                  # Log files
|   +-- reports/               # Generated reports
+-- plugins/                   # Custom plugins
```

Appendix C Release History

C.1 Version 2.0.0 (February 25, 2025)

Version 2.0.0 expanded the framework from 36 to 105 modules and hardened the codebase. The release highlights are:

- 69 new modules across all seven categories, including database exploits (MySQL, MSSQL, PostgreSQL, Oracle, Redis), network exploits (FTP, Telnet, RDP, VNC, SMTP, SNMP, LDAP, NFS, AD, Wireless, Cloud), and advanced reconnaissance (OSINT, SCADA, WiFi, Blockchain, Firmware, Supply Chain, Threat Intel).
- New post-exploitation coverage: lateral movement, container escape, memory forensics, and anti-forensics.
- New evasion capabilities: AV/AMSI bypass, DNS tunneling, and JA3 randomization.
- New payload tooling: multi-handler, shellcode generator, and macro builder.
- New auxiliary scanners: FTP, HTTP, SSH, SSL, SNMP, NetBIOS, RDP, and IPMI.
- Quality: fixed 88+ bare except clauses, added check() to all module classes, suppressed the paramiko deprecation warning, fixed duplicate banner display, verified all 153 Python files with zero syntax errors, and validated all 105 modules and the 72 CLI commands.

C.2 Version 1.0.0

The initial release established the core framework: the dynamic module loader with 36 modules, the interactive CLI with tab completion, the REST API and web dashboard, the auto-exploitation engine, and the encrypted C2 infrastructure.